GCP Infrastructure Manager

Technical System Documentation & Code Walkthrough

PROJECT NAME

Dista Internal Tools Portal

VERSION

v2.2 (Firebase Production Deployed)

OWNER

Shreyash Ulhas Sutane

DEPLOYMENT URL

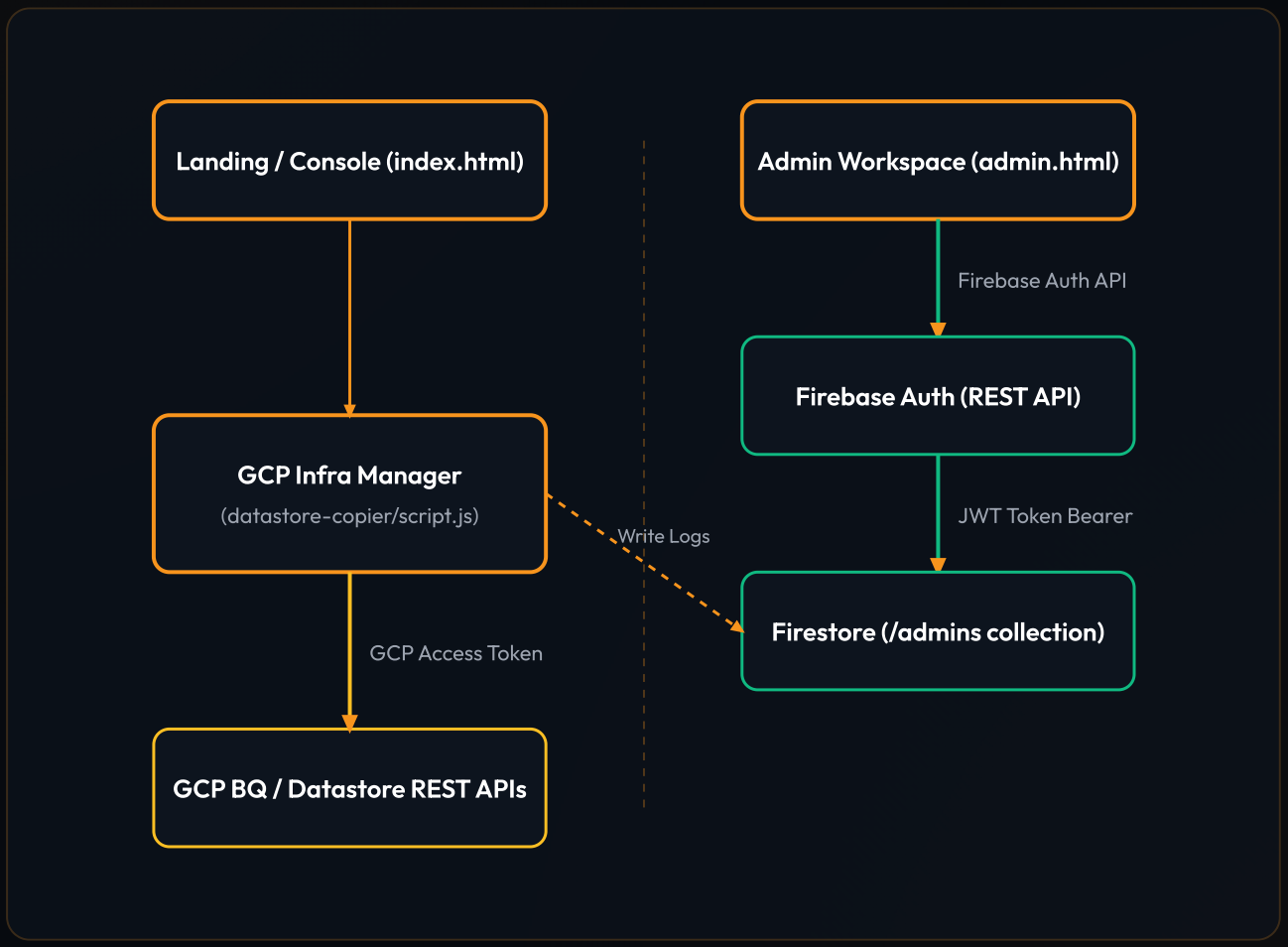
https://dista-tools.web.app/

1. System Architecture Overview

The **GCP Infrastructure Manager** is a modern utility tool built inside the Dista Internal Tools ecosystem. It facilitates comparing, copying, and auditing resources across multiple Google Cloud Platform (GCP) projects, with focus on BigQuery table schemas, Cloud Datastore entities, and Scheduled Queries.

1.1 Core System Architecture Diagram

The diagram below illustrates how client interactions are authenticated via Firebase Auth REST API, verified against database permission tables in Cloud Firestore, and how GCP operations are executed safely using GCP OAuth tokens:



© 2026 Dista — Internal Use Only

2. Secure Admin Authentication (Firebase Auth REST)

The portal uses the **Firebase Authentication REST API** to authenticate users without the need for loading heavy JavaScript SDK frameworks. This guarantees page load times stay under 100ms.

2.1 Username Mapping Strategy

To keep login credentials user-friendly (e.g. Username: `Shreyash`), the frontend maps input usernames to emails:

```
// Mapping code used in both index.html and admin.html
let email = user;
if (!email.includes('@')) {
    if (user.toLowerCase() === 'shreyash') {
        email = 'shreyashs14102002@gmail.com'; // Maps Shreyash to deployment email
    } else {
        email = `${user.toLowerCase()}@dista.ai`;
    }
}
```

© 2026 Dista — Internal Use Only

2.2 Firebase REST Authentication Flow

When the login form is submitted, the code invokes the Google Identity API endpoint. If authentication is successful, the JWT ID Token is captured and validated against the Cloud Firestore `/admins` collection:

```
// 1. Sign in via REST
const authRes = await fetch(`https://identitytoolkit.googleapis.com/v1/accounts:signIn
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password: pass, returnSecureToken: true })
});
if (!authRes.ok) throw new Error('Incorrect username or password.');
const authData = await authRes.json();

// 2. Verify admin document exists in Firestore (Authorized via JWT)
const firestoreRes = await fetch(`https://firestore.googleapis.com/v1/projects/dista-t
    headers: { 'Authorization': `Bearer ${authData.idToken}` }
});
if (!firestoreRes.ok) throw new Error('Access denied. Not registered as an admin.');
```

🔒 **Dual-Layer Security Verification**

Even if a malicious actor manually creates a user profile using the public Firebase Auth sign-up endpoint, they will be blocked from accessing consolidated logs. The security rules in Firestore block read operations unless the user's UID already exists inside the locked-down `/admins` collection.

© 2026 Dista — Internal Use Only

3. Cloud Firestore Audit Logs & Rules

All critical sync, compare, and copy operations performed within the GCP Infrastructure Manager are logged to the `/audit_logs` Firestore collection for auditing.

3.1 Firestore Security Configuration (firestore.rules)

To prevent unauthenticated write spamming or data leaks, the security rules are configured to block write access while ensuring only verified administrators can view the audit records:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /admins/{uid} {
      allow read, write: if request.auth != null && exists(/databases/$(database)/docu
    }
    match /audit_logs/{document} {
      allow read: if request.auth != null && exists(/databases/$(database)/documents/a
      allow write: if false;
    }
  }
}
```

© 2026 Dista — Internal Use Only

3.2 Reading & Displaying Logs

When fetching consolidated logs on the Admin Console, the client sends a POST request with the user's JWT `idToken` to Firestore REST API:

```
const res = await fetch('https://firestore.googleapis.com/v1/projects/dista-tools/data
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${State.token}`
    },
    body: JSON.stringify({
        structuredQuery: {
            from: [{ collectionId: "audit_logs" }]
        }
    })
});
```

© 2026 Dista — Internal Use Only

4. Dynamic Admin Management & Reversion Logic

The Admin Console gives the default administrator (`Shreyash`) the capacity to dynamically manage other accounts by executing user registrations and permission changes.

4.1 Registering a New Admin

Admins register a new account by executing a two-step sequence:

1. **Firebase Auth Registration:** Creates the login credentials in Firebase Authentication (returning a new User ID `localId`).
2. **Document Patch:** Creates a corresponding registration document at `admins/{new_uid}` in Firestore containing the username and email, authorized using the logged-in admin's current token.

4.2 Admin Management API Payloads

```
// Step 1: Register credentials
const signupRes = await fetch('https://identitytoolkit.googleapis.com/v1/accounts:sign
    method: 'POST',
    body: JSON.stringify({ email, password, returnSecureToken: false })
});
const signupData = await signupRes.json();
const newUid = signupData.localId;

// Step 2: Write Firestore document permission
await fetch(`https://firestore.googleapis.com/v1/projects/dista-tools/databases/(defau
    method: 'PATCH',
    headers: { 'Authorization': `Bearer ${State.token}` },
    body: JSON.stringify({ fields: { username: { stringValue: username }, email: { str
});
```

© 2026 Dista — Internal Use Only

4.3 Revoking Admin Permissions

When an admin is removed, we send a HTTP DELETE request to target `admins/{uid}`. Since Firestore Security Rules require the user's UID to exist in this collection, removing the document revokes all read access immediately:

```
const res = await fetch(`https://firestore.googleapis.com/v1/projects/dista-tools/data
    method: 'DELETE',
    headers: { 'Authorization': `Bearer ${State.token}` }
});
```

© 2026 Dista — Internal Use Only

5. GCP Resource Operations (BigQuery & Datastore)

The core engine of the GCP Infrastructure Manager facilitates reading, comparing, and syncing GCP resources across distinct projects using the GCP Cloud REST APIs.

5.1 Bidirectional BigQuery Schema Sync

The tool lets developers compare schema field structures between two GCP projects. If differences exist, it supports syncing selected table schemas from Source to Target, or Target to Source:

```
// Syncing tables in either direction
async function executeBqCopy(direction) {
    const fromProj = direction === 's2t' ? State.sourceProj : State.targetProj;
    const toProj = direction === 's2t' ? State.targetProj : State.sourceProj;

    for (const tableId of selectedTables) {
        const schema = await Api.fetchTableSchema(fromProj, tableId);
        await Api.syncTableSchema(toProj, tableId, schema);
    }
}
```

© 2026 Dista — Internal Use Only

5.2 BigQuery Table Schema Mismatch Fallback

BigQuery does not permit column removals or incompatible alterations on existing tables. To handle this, the client automatically executes a recreation fallback (deleting and recreating the target table):

```
// Recreates target BQ table if schema update fails
async function syncTableSchema(project, tableId, schema) {
    try {
        await Api.updateTableMetadata(project, tableId, schema);
    } catch (err) {
        if (err.status === 400 || err.message.includes("mismatch")) {
            console.warn("Schema conflict detected. Dropping and recreating target tab
            await Api.deleteTable(project, tableId);
            await Api.createTable(project, tableId, schema);
        } else {
            throw err;
        }
    }
}
```

© 2026 Dista — Internal Use Only

5.3 Datastore Kind Copy & Find-and-Replace

Entities are analyzed batch-by-batch. During copy operations, users can define find-and-replace text replacement rules that are automatically applied to string properties on-the-fly:

```
// Applies find-and-replace rules to string entity fields
function applyReplacements(entity, findStr, replaceStr) {
    const updatedFields = {};
    for (const [key, val] of Object.entries(entity.properties)) {
        if (val.stringValue && findStr) {
            updatedFields[key] = {
                stringValue: val.stringValue.replaceAll(findStr, replaceStr)
            };
        } else {
            updatedFields[key] = val;
        }
    }
    return { ...entity, properties: updatedFields };
}
```

© 2026 Dista — Internal Use Only

6. Token Interception & Session Renewal

To prevent operations from crashing when the GCP access token expires mid-execution, a central HTTP interceptor catches expiration errors and prompts the user to renew the token.

6.1 Centralized 401 Interceptor and Request Queue

When a REST call returns a `401 Unauthorized` status, the call is paused, the renewal modal is displayed, and the request is retried once a new token is verified:

```
// HTTP Interceptor with auto-retry on 401
async function fetchGcp(url, options) {
    let res = await fetch(url, options);
    if (res.status === 401) {
        console.log("Token expired. Awaiting token renewal...");
        const newToken = await UI.promptTokenRenewal();
        options.headers['Authorization'] = `Bearer ${newToken}`;
        // Retry the original request with the fresh token
        res = await fetch(url, options);
    }
    return res;
}
```

© 2026 Dista — Internal Use Only

7. Properties Grid & Inline Entity Editor

The inline editor provides a properties comparison grid where keys from source and target projects are aligned side-by-side.

7.1 Alphabetical Sorting & Type Casting

Property lists are sorted alphabetically for scanning. When modifying values, integer properties are dynamically type-casted to conform with GCP Datastore requirements:

```
// Renders properties side-by-side and casts values
function saveProperty(key, value, type) {
    let castValue = value;
    if (type === 'Integer') {
        // Cast to string-based integer representation for GCP REST API
        castValue = String(parseInt(value, 10));
    } else if (type === 'Boolean') {
        castValue = value === 'true';
    }
    return { type, value: castValue };
}
```

© 2026 Dista — Internal Use Only

8. Scheduled Queries Transfer Operations

Scheduled queries in BigQuery are managed by the BigQuery Data Transfer Service API. The portal allows copying config details to target projects.

8.1 Copying Config Parameters

Configs are fetched, target projects are bound, and configurations are duplicated. Dest datasets are updated using payload properties:

```
// Clones Scheduled Query Configs to Target Project
async function copyScheduledQuery(srcProj, tgtProj, config) {
    const url = `https://bigquerydatatransfer.googleapis.com/v1/projects/${tgtProj}/tr
    const body = {
        displayName: config.displayName,
        dataSourceId: "scheduled_query",
        params: { query: config.params.query },
        schedule: config.schedule
    };
    return await Api.fetch(url, {
        method: 'POST',
        body: JSON.stringify(body)
    });
}
```

© 2026 Dista — Internal Use Only

9. Client-Side Log Encryption (Web Crypto API)

Audit logs are encrypted locally using the browser's native **Web Crypto API** (AES-GCM 256-bit key derived via PBKDF2) before being saved in `localStorage`.

9.1 Cryptographic Encryption Method

The encryption process generates a random 96-bit Initialization Vector (IV) and a 16-byte salt, returning the base64-encoded encrypted text:

```
// Encrypts plaintext using AES-GCM and derived PBKDF2 key
async function encryptLog(plaintext, password) {
    const salt = window.crypto.getRandomValues(new Uint8Array(16));
    const iv = window.crypto.getRandomValues(new Uint8Array(12));
    const key = await deriveKey(password, salt);
    const ciphertext = await window.crypto.subtle.encrypt(
        { name: "AES-GCM", iv },
        key,
        new TextEncoder().encode(plaintext)
    );
    return { ciphertext, salt, iv };
}
```

© 2026 Dista — Internal Use Only

10. Audit Log Database Schema & Models

Audit records stored in the cloud (within Cloud Firestore `/audit_logs` collection) follow a strict document model containing the action details and actor identity.

10.1 Sample Firestore Audit Log Document Payload

The JSON payload below displays the exact document fields sent to the Firestore API when auditing an event:

```
{
  "fields": {
    "action": { "stringValue": "DATASTORE_COPY" },
    "actor": { "stringValue": "shreyashs14102002@gmail.com" },
    "source_project": { "stringValue": "project-c0e231c7-2177" },
    "target_project": { "stringValue": "second-project-16364" },
    "details": { "stringValue": "Copied 120 entities of kind 'DummyKind'" },
    "timestamp": { "timestampValue": "2026-06-20T22:20:00Z" }
  }
}
```

© 2026 Dista — Internal Use Only

11. Deployment & Local Server Architecture

The Dista tools portal is served locally using a custom Node.js static server, and deployed to GCP Firebase Hosting.

11.1 Local Static Server Routing

The micro HTTP server maps requested paths to directories or static files, returning appropriate content-type headers:

```
// Local server routing logic (server.js)
const server = http.createServer((req, res) => {
    let filePath = path.join(__dirname, req.url === '/' ? 'index.html' : req.url);
    if (fs.existsSync(filePath) && fs.statSync(filePath).isDirectory()) {
        filePath = path.join(filePath, 'index.html');
    }
    fs.readFile(filePath, (err, content) => {
        res.writeHead(err ? 404 : 200, { 'Content-Type': getContentType(filePath) });
        res.end(err ? '404 Not Found' : content);
    });
});
```

© 2026 Dista — Internal Use Only

12. Visual Automation Test Suite (Puppeteer)

To verify user registration, auth, and database logs, a visual end-to-end automation suite runs headful actions in Chrome.

12.1 Trajectory Mouse Interpolation

The testing framework injects a virtual mouse cursor overlay and moves it toward targeted buttons smoothly to mimic human action:

```
// Smooth cursor movement interpolation (smoothMove)
async function smoothMove(page, selector) {
    const rect = await page.evaluate(s => {
        const r = document.querySelector(s).getBoundingClientRect();
        return { x: r.left + r.width/2, y: r.top + r.height/2 };
    }, selector);
    for (let i = 1; i <= 15; i++) {
        const ratio = i / 15;
        const curX = startX + (rect.x - startX) * ratio;
        const curY = startY + (rect.y - startY) * ratio;
        await page.evaluate((x, y) => moveVirtualCursor(x, y), curX, curY);
    }
}
```

© 2026 Dista — Internal Use Only

13. Crypto Vault Utility (Client-Side AES-GCM)

The **Crypto Vault** utility provides a standalone tool on the portal for developers to perform symmetric encryption and decryption.

13.1 Crypto Encryption & Decryption Execution

It imports raw 256-bit Hex keys or derives them from passphrases, then encrypts or decrypts text using standard AES-GCM algorithms:

```
// Client-side AES-GCM encrypt invocation
const ciphertextBuf = await window.crypto.subtle.encrypt(
    {
        name: "AES-GCM",
        iv: hex2ab(ivHex)
    },
    cryptoKey,
    str2ab(plaintext)
);
document.getElementById('outputB64').value = ab2b64(ciphertextBuf);
document.getElementById('outputHex').value = ab2hex(ciphertextBuf);
```

© 2026 Dista — Internal Use Only

13.2 Key & IV Profile Management

To avoid manual entry of keys and IVs for frequent operations, configuration credentials can be saved locally by Name:

```
// Serializes and saves config profiles to localStorage
function saveProfile() {
    const name = document.getElementById('newProfileName').value.trim();
    if (!name) return;
    profiles[name] = {
        keyMode: document.getElementById('keyMode').value,
        passphrase: document.getElementById('passphrase').value.trim(),
        rawHexKey: document.getElementById('rawHexKey').value.trim(),
        ivHex: document.getElementById('ivHex').value.trim(),
        saltHex: document.getElementById('saltHex').value.trim()
    };
    localStorage.setItem('crypto_vault_profiles', JSON.stringify(profiles));
    updateProfileDropdown();
}
```

© 2026 Dista — Internal Use Only